# DATA 211 – Introduction to Data Science
# Topic 4: Data Preprocessing

Question bank: 40 multiple choice + 10 programming

Questions from DATA 211 / ISE 291 old exams and quizzes (AI_Work). **[new]** marks new questions written in the same style.

## Part A: Multiple choice

### Q1.

Sol-Practice Questions Topic5

What is data acquisition?

(A) Acquiring the data for analysis

(B) Sorting the data

(C) Transforming the data

(D) Cleaning the data

**Answer:** (A). Data acquisition is the act of acquiring data for analysis.

My answer

### Q2.

Topic 4 and 5 (Quiz 2, Sec 4)

What is the direct approach in data acquisition?

(A) It is a process of identifying sources/vendors that provide the data based on specifications

(B) It is a process that converts analog and/or real world data into digital data that can be processed for further analysis

(C) It is a process of cleaning and processing the acquired data

(D) It is a process of storing the acquired data in a database

**Answer:** (B). The direct approach converts real-world (analog) data into digital data, using a Data Acquisition System (DAS). (A) is the indirect approach.

My answer

### Q3.

Topic 4 and 5 (Quiz 02, Sec 09)

Getting a dataset from an online repository about the average number of people visiting Jarir bookstore between 2pm and 3pm is a _____ data acquisition approach.

(A) direct

(B) indirect

(C) hybrid

(D) None of the answers

**Answer:** (B). Obtaining data from an existing source/vendor (a repository) is the indirect approach.

My answer

**Q4.** Sample Mid

You are conducting data science research on consumer behavior for a retail company. Which of the following is an indirect approach for data collection?

(A) Surveying the company customers

(B) Interviewing company customers

(C) Installing a Data Acquisition System

(D) Using data from public data repositories (e.g. Kaggle)

**Answer:** (D). Taking data from existing sources (open repositories) is indirect; surveys, interviews and a DAS collect the data directly.

My answer

**Q5.** Topic 4 and 5 (Quiz 02, Sec 09)

What is the correct code to read a file called `data01.tsv` whose rows' elements are separated by a tab `'\t'`?

(A) `pd.read_tsv("data01.tsv", delimiter='\t')`

(B) `pd.read_tsv("data01.tsv")`

(C) `pd.read_tsv("data01.tsv", delimiter=',')`

(D) `pd.read_csv("data01.tsv", delimiter='\t')`

**Answer:** (D). Pandas has no `read_tsv`; a TSV file is read with `read_csv` and a tab delimiter.

My answer

**Q6.** Sol-Practice Questions Topic5

What makes data dirty?

(A) The occurrence of noise in data

(B) The data are unstructured

(C) The data are not in the .csv or .xlsx format

(D) The data are raw and need further processing to become information

**Answer:** (A). Dirty data is incomplete, noisy or inconsistent; format or structure alone does not make data dirty.

My answer

**Q7.** Topic 4 and 5 (Quiz 02, Sec 07)

Assume the ages of primary school students are [10, 9, 8, $-9$, 11]. What is the possible problem in the dataset?

(A) Incomplete

(B) Noisy

(C) Inconsistency

(D) No problem (all values are integers)

**Answer:** (B). An age of $-9$ is an error value (like Salary $= -10$), i.e. noisy data.

My answer

**Q8.** Sample Mid

Consider the following data. Which record has noisy data?

| | ID | Age | Salary | Gender | Job Title |
|---|---|---|---|---|---|
| 0 | 201223 | 23.0 | 20000.0 | M | Direct |
| 1 | 300224 | $-30.0$ | 15000.0 | F | Consultant |
| 2 | 202225 | NaN | 18000.0 | M | Employee |
| 3 | 303226 | 26.0 | NaN | F | Employee |
| 4 | 200027 | 55.0 | 31000.0 | M | Director |

(A) Record with index 0 (first record)

(B) Records with indexes 0 and 4

(C) Records with indexes 2 and 3

(D) Record with index 1

**Answer:** (D). Age $= -30$ is an erroneous value (noise). Records 2 and 3 have missing values (incomplete), not noise.

My answer

**Q9.** Q2-F-Key

Which of the following is an example of inconsistent data?

(A) Age $= -5$

(B) Salary = NaN

(C) Height = 180 cm

(D) Gender = ”Male” and ”M”

**Answer:** (D). Two different codes for the same value is a discrepancy in codes/names (inconsistency). (A) is noise, (B) is incomplete.

My answer

**Q10.** Midterm Practice Questions 2

Which of the following is an example of inconsistent data types regarding a column named `salary`?

(A) Having missing values in the salary column

(B) Having extremely high or low values (outliers) in the salary column

(C) Having some values as numbers and some as symbols in the salary column

(D) Having all values as integers

**Answer:** (C). Mixing numbers and symbols in one column is a type inconsistency; (A) is incomplete data and (B) is noise.

My answer

**Q11.** 223 Midterm

Upper and lower whisker limits can be used to identify _____.

(A) Noisy data

(B) Missing values

(C) Inconsistent data type

(D) Normality of the data

**Answer:** (A). Values outside the whisker limits are outliers, which are a kind of noisy data.

My answer

**Q12.** Sample Mid

Which of the following is considered as data cleaning in data analytics projects?

(A) Scaling the data using standard scaler

(B) Encoding the data using one-hot encoding

(C) Discretizing the data

(D) Resolving inconsistency

**Answer:** (D). Data cleaning = fill in missing values, smooth noisy data, identify/remove outliers and resolve inconsistencies. The others are transformation/discretization.

My answer

### Q13.

Midterm Practice Questions 2

Which of the following is NOT a step in data cleaning?

(A) Removing outliers

(B) Handling missing values

(C) Creating new features

(D) Correcting data types

**Answer:** (C). Creating new features is attribute/feature construction, which is part of data transformation, not cleaning.

My answer

### Q14.

F08 Quiz02

What are the two key steps in data preprocessing?

(A) Cleaning and visualization

(B) Scrubbing and modeling

(C) Cleaning and transformation

(D) Exploring and exporting

**Answer:** (C). Among the options, only cleaning and transformation are both major preprocessing tasks.

My answer

### Q15.

F08 Quiz02

Which is not a preprocessing step?

(A) Data visualization

(B) Handling missing values

(C) Encoding categorical variables

(D) Feature scaling

**Answer:** (A). Visualization is used for exploration; handling missing values, encoding and scaling prepare the data.

My answer

**Q16.** Sol-Practice Questions Topic5

Filtering noisy data is:

(A) Finding missing data

(B) Applying One-Hot-Encoding

(C) Transforming the data

(D) Identifying outliers and removing them from the database

**Answer:** (D). Noise includes errors and outliers; filtering it means identifying and removing those outliers.

My answer

**Q17.** Topic 4 and 5 (Quiz 02, Sec 09)

What does discretization mean as a transformation technique?

(A) Grouping/Mapping

(B) Filtering

(C) Scaling

(D) Encoding

**Answer:** (A). Discretization groups (maps) numerical values into a small number of intervals or categories.

My answer

**Q18.** Sample Mid

In a pandas DataFrame, you have a field named ”Total Scores” with numerical values representing the total scores of all the students in a class. The process of converting the total scores into letter grades like A+, A, B+, B, C+, C, D+, D, F is called:

(A) Discretization

(B) Normalization

(C) Scaling

(D) Standardization

**Answer:** (A). Mapping a numerical range into a few grade categories is discretization.

My answer

**Q19.** 223 Midterm

Given the field `age` with values in years, the process of converting the age values from years into age categories (child, adult, older adult) is called _____.

(A) Scaling

(B) Normalization

(C) Discretization

(D) Imputation

**Answer:** (C). Grouping numerical ages into categories is discretization.

My answer

**Q20.** Sample Mid

Which of the following is **not** used to handle missing values?

(A) Replacing missing values by the mean and mode

(B) Removing noisy data

(C) Replacing missing values with a global constant

(D) Deleting rows or columns containing missing values

**Answer:** (B). Removing noise handles errors/outliers. The other three are the slide methods (attribute mean/mode, global constant, ignore the tuple).

My answer

**Q21.** Sol-Practice Questions Topic5

What is the purpose of data imputing?

(A) Filtering noisy data

(B) Formatting data into usable format

(C) To handle missing data

(D) To delete duplicate data

**Answer:** (C). Imputation fills in missing values automatically (e.g. with the attribute mean or mode).

My answer

### Q22.

Sample Mid

For a pandas DataFrame `df`, what does the following code return?

```
df.isna().sum()
```

(A) A pandas Series of the number of NaN in each row

(B) A pandas Series of True/False regarding the existence of NaN in each row

(C) A pandas Series of the number of NaN in each column

(D) A pandas Series of True/False regarding the existence of NaN in each column

**Answer:** (C). `isna()` gives a Boolean frame; `sum()` adds down each column, giving the count of missing values per column.

My answer

### Q23.

Quiz_2 (docx)

To replace missing values in column `'salary'` with the mean, use:

(A) `df['salary'].replace(mean)`

(B) `df['salary'].fillna(df['salary'].mean())`

(C) `df['salary'].fill(df['salary'].mean())`

(D) `df['salary'].apply(mean)`

**Answer:** (B). `fillna` fills the NaN values with the given value, here the attribute mean.

My answer

### Q24.

Sample Mid

What is the correct explanation for the following code?

```
df = pandas.DataFrame([[2, None], [None, 'N'], [3, 'N'], [1, 'Y']], columns=['A', 'B'])
df['C'] = df['A'].fillna(df['A'].mean())
```

(A) Change only the name of column A to C

(B) Change the name of column A to C, and replace NaN by 2

(C) Add a new column named C to `df` with the values of column A after replacing NaN by 2. No change in column A

(D) Add a new column named C to `df` with the values of column A after replacing None by 2. NaN in column A is also replaced by 2

**Answer:** (C). Mean of A $= (2+3+1)/3 = 2$. `fillna` returns a new Series that is stored in C; A itself keeps its NaN.

My answer

**Q25.** Sample Mid

Given a DataFrame `df`, what is the correct explanation for the following?

```
c_null = df.loc[:, df.isna().any()].columns
for c in c_null:
    value = df[c].mean() if df[c].dtype != 'object' else df[c].mode()[0]
    df[c].fillna(value, inplace=True)
```

(A) It replaces the NaN values in all the columns by the mean

(B) It replaces the NaN values in `df` by either the mean or the mode depending on the column data type

(C) It replaces the NaN values in all the columns by the mode

(D) It replaces the NaN values in the numerical columns by the mode

**Answer:** (B). For each column with missing values: numeric columns get the mean, object (categorical) columns get the mode.

My answer

**Q26.** Q2-F-Key

What will be the result of the following code?

```
import pandas as pd
df = pd.DataFrame({'Category': ['A','A','C',None,'B'], 'Value': [10,20,None,30,40]})
df['Category'].fillna(df['Category'].mode()[0], inplace=True)
```

(A) All NaN values in the `'Category'` column will be replaced with 25

(B) NaN values in the `'Category'` column will remain unchanged

(C) All NaN values in the `'Category'` column will be replaced with the value `'A'`

(D) All NaN values in the `'Category'` column will be replaced with NaN

**Answer:** (C). The mode of Category is `'A'` (it appears twice), so the missing entry becomes `'A'`.

My answer

**Q27.** Sample Mid

The following data represent the scores of 5 students in 3 quizzes (Q1, Q2 and Q3). Some students missed one quiz, and the instructor decided to give those students the mean of their other two quizzes for the missed quiz.

| | Before imputation | | | | After imputation | | |
|---|---|---|---|---|---|---|---|
| | Q1 | Q2 | Q3 | | Q1 | Q2 | Q3 |
| Mohammed | NaN | 5.0 | 7.5 | Mohammed | 6.25 | 5.0 | 7.5 |
| Hassan | 10.0 | 9.0 | 10.0 | Hassan | 10.00 | 9.0 | 10.0 |
| Ali | 8.0 | 10.0 | 9.0 | Ali | 8.00 | 10.0 | 9.0 |
| Fahd | 7.0 | 6.0 | 9.0 | Fahd | 7.00 | 6.0 | 9.0 |
| Saleh | 10.0 | NaN | 10.0 | Saleh | 10.00 | 10.0 | 10.0 |

What is the correct code to impute the score of the missed quiz with the mean of the other two quizzes?

(A)

```
for x in df.columns:
    m = df.loc[x, :].mean()
    df.loc[x, :] = df.loc[x, :].fillna(m, axis=0)
```

```
for x in df.columns:
    m = df.loc[:, x].mean()
    df.loc[:, x] = df.loc[:, x].fillna(m, axis=0)
```

(B)

```
for x in df.index:
    m = df.loc[:, x].mean()
    df.loc[:, x] = df.loc[:, x].fillna(m, axis=0)
```

```
for x in df.index:
    m = df.loc[x, :].mean()
    df.loc[x, :] = df.loc[x, :].fillna(m, axis=0)
```

**Answer:** (D). The mean must be taken across each row (student): loop over `df.index` and use `df.loc[x, :]`. Mohammed: $(5 + 7.5)/2 = 6.25$.

My answer

## Q28.

Sample Mid

For a pandas DataFrame `df`, what is the correct explanation of the following code snippet?

```
df.dropna(axis=1, thresh=len(df.index)-3, inplace=True)
```

(A) Remove all columns containing more than 3 NaN

(B) Remove all rows containing more than 3 NaN

(C) Keep all columns which contain more than 3 non-NaN

(D) Keep all rows which contain more than 3 non-NaN

**Answer:** (A). With `axis=1` a column is kept only if it has at least $n - 3$ non-NaN values ($n$ = number of rows), i.e. columns with more than 3 NaN are dropped.

My answer

### Q29.

223 Midterm

A DataFrame `df` has 8 rows. Its column `Quiz1` has 2 missing values, and all the values in the column `Final` are null (missing); the other columns have no missing values. Which of the following codes will drop **only** the column `Final` from the DataFrame?

(A) `df.dropna(axis=1, how='all', thresh=8, inplace=True)`

(B) `df.dropna(axis=0, how='all', thresh=8, inplace=True)`

(C) `df.dropna(axis=1, how='all', inplace=True)`

(D) `df.dropna(axis=0, how='all', inplace=True)`

**Answer:** (C). `axis=1, how='all'` drops only columns whose values are all missing. With `thresh=8`, `Quiz1` (6 non-null values) would also be dropped (newer pandas raises an error when `how` and `thresh` are both given). `axis=0` works on rows.

My answer

### Q30.

Midterm Practice Questions 2

The data represent the scores of 5 students in 3 quizzes. The Q3 column has a float representation mixed with string characters like a comma because of formatting issues (`df.info()` shows Q3 as `object`).

| | Name | Q1 | Q2 | Q3 |
|---|---|---|---|---|
| 0 | Mohammed | NaN | 5.0 | 7,5 |
| 1 | Hassan | 10.0 | 9.0 | 10 |
| 2 | Ali | 8.0 | 10.0 | 9 |
| 3 | Fahd | 7.0 | 6.0 | 9 |
| 4 | Saleh | 10.0 | NaN | 10 |

Identify the right code to safely convert Q3 to float numbers.

(A) `df['Q3']=df['Q3'].apply(lambda x: float(x))`

(B) `df['Q3']=df['Q3'].applymap(lambda x: float(str(x).replace(',',''))`

(C) `df['Q3']=df['Q3'].applymap(lambda x: float(x))`

(D) `df['Q3']=df['Q3'].apply(lambda x: float(str(x).replace(',','.'))`

**Answer:** (D). A Series uses `apply` (not `applymap`); `'7,5'` must become `'7.5'` before `float`. (A) fails on `'7,5'`.

My answer

### Q31.

223 Midterm

Which of the following is the correct method to resolve the type inconsistency in a numeric column named `"salary"`, where the values are formatted as `"SAR 12345"`?

(A) `df['salary']=df['salary'].apply(lambda x: x.replace("SAR",""))`

(B) `df['salary']=df['salary'].apply(lambda x: int(x.replace("SAR","")))`

(C) `df['salary']=df['salary'].replace("SAR","")`

(D) `df['salary']=df['salary'].applymap(lambda x: int(x.replace("SAR","")))`

**Answer:** (B). Remove the text and cast to `int` for each element. (A) leaves strings, (C) replaces only whole-cell matches, (D) `applymap` is not a Series method.

My answer

### Q32.

223 Midterm

In a DataFrame `df`, the column `HWs` contains [80, 9, 7, 10, 8, 70, 90, 7], where some values were entered out of 100 instead of out of 10. Which of the following codes can manipulate the data in the column `HWs` such that all values of `HWs` are out of 10?

(A) `df['HWs']=df['HWs'].applymap(lambda x: x/10 if x>10)`

(B) `df['HWs']=df['HWs'].applymap(lambda x: x/10 if x>10 else x)`

(C) `df['HWs']=df['HWs'].apply(lambda x: x/10 if x>10)`

(D) `df['HWs']=df['HWs'].apply(lambda x: x/10 if x>10 else x)`

**Answer:** (D). A conditional expression in a lambda needs an `else`, and a Series uses `apply`. Result: [8, 9, 7, 10, 8, 7, 9, 7].

My answer

### Q33.

Sol-Practice Questions Topic5

The following code will:

```
selected_cols = df.select_dtypes(include='number').columns
df[selected_cols] = df[selected_cols].applymap(lambda x: x if x>=0 else abs(x))
```

(A) Compute the absolute value of all the values

(B) Replace negative values by their absolute values

(C) Make all the values negative

(D) Compute the absolute value of all positive values

**Answer:** (B). Non-negative values are kept as they are; only negative (noisy) values are replaced by their absolute values.

My answer

### Q34.

Sol-Practice Questions Topic5

Min-Max normalization is used to:

(A) Transform data to mean 0 and standard deviation 1

(B) Transform data to minimum 0 and maximum 1

(C) Find the minimum and maximum values

(D) Sort the data in ascending order

**Answer:** (B). With the default range [0, 1], min-max maps the minimum to 0 and the maximum to 1.

My answer

### Q35.

Topic 4 and 5 (Quiz 02, Sec 09)

Min-Max and Z-Score scalers are examples of data _____.

(A) Reduction

(B) Discretization

(C) Normalization

(D) Conversions

**Answer:** (C). Both rescale numerical values, i.e. normalization (a data transformation).

My answer

### Q36.

223 Midterm

Given a column called `age` in a DataFrame `df`, which of the following will scale the age column such that its maximum value is 1 and its minimum value is 0?

(A)

```
from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler()
scaler.fit(df[['age']])
df['age'] = scaler.transform(df[['age']])
```

```
from sklearn.preprocessing import MinMaxScaler
scaler.fit(df[['age']])
df['age'] = scaler.transform(df[['age']])
```

(B)

```
from sklearn.preprocessing import MinMaxScaler
MinMaxScaler()
scaler.fit(df[['age']])
df['age'] = scaler.transform(df[['age']])
```

```
from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler()
df['age'] = scaler.transform(df[['age']])
scaler.fit(df[['age']])
```

**Answer:** (A). The scaler must be created and assigned to a variable, then fitted, then used to transform. (B) and (C) never define `scaler`; (D) transforms before fitting.

My answer

**Q37.** Exam (11) – Quiz 2 T251 key

You have a pandas DataFrame `df` with only numeric columns, and you want to apply Min-Max scaling normalization to specific columns (`'A'` and `'B'`) while leaving other columns unchanged.

```
import pandas as pd
df = pd.DataFrame({
    'A': [10, 20, 30, 40],
    'B': [5.0, 15.5, 25.0, 35.5],
    'C': [5.0, 15.5, 25.0, 35.5]
})
```

Which of the following code snippets correctly achieves this using `MinMaxScaler` from `sklearn.preprocessing`?

**(A)**
```
scaler = MinMaxScaler()
df[['A', 'B']] = scaler.fit_transform(df[['A', 'B']])
```

```
scaler = MinMaxScaler()
df = df.apply(scaler.fit_transform)
```

**(B)**
```
scaler = MinMaxScaler()
df[['A', 'B']] = df[['A', 'B']].apply(scaler.fit_transform)
```

```
scaler = MinMaxScaler()
df['A', 'B'] = scaler.fit_transform(df['A', 'B'])
```

**Answer:** (A). The scaler needs a 2-D selection `df[['A','B']]`. (B) and (C) pass 1-D Series to the scaler (error), (D) looks up a single column named `('A','B')` (KeyError).

My answer

**Q38.** Quiz2AnswerMS (T251)

Calculate the median of the following data after converting the data with the Min-max scaler.

$$D = [1, 3, 5, 7, 9], \qquad \hat{x} = \frac{x_i - x_{\min}}{x_{\max} - x_{\min}}$$

**(A)** 0.75

(B) 1

(C) 0

(D) 0.5

**Answer:** (D). Scaled data $= [0, 0.25, 0.5, 0.75, 1]$, so the median is 0.5.

My answer

**Q39.** Sample Mid

Given data with a column called `person_age`, where the age values range from 20 to 60. Applying the MinMaxScaler formula below, what will be the scaled age value if the person's age is 40?

$$\text{scaled value} = \frac{x_i - \min}{\max - \min}$$

(A) 0.5

(B) 1.0

(C) 0

(D) 1.5

**Answer:** (A). $(40 - 20)/(60 - 20) = 20/40 = 0.5$.

My answer

**Q40.** Sample Mid

Suppose that the columns "Q1, Q2, Q3, Q4" contain the quiz scores of all the students in a class. The process of creating a new column containing the sum of the best three quiz scores is called:

(A) One Hot Encoding

(B) Normalization

(C) Feature Construction

(D) Min-Max Normalization

**Answer:** (C). A new attribute built from existing ones is attribute/feature construction (like BMI from weight and height).

My answer

# Part B: Programming

## Q41.

Set_2 (Midterm Set-2), C2

Column ENGL101 scores have been entered out of 25. Scale the ENGL101 scores to be out of 100 and make the change permanent in the DataFrame. Use:

```
df = pd.DataFrame({'STU ID': [202010100, 202010357, 202010750, 202011111, 202011378],
                   'ENGL101': [12, 20, 25, 18, 9]})
```

**Answer:**

```
import pandas as pd
df = pd.DataFrame({'STU ID': [202010100, 202010357, 202010750, 202011111, 202011378],
                   'ENGL101': [12, 20, 25, 18, 9]})
# out of 25 -> out of 100: multiply every score by 100/25 = 4
df['ENGL101'] = df['ENGL101'].apply(lambda x: 4 * x)
print(df)
# Note: MinMaxScaler((0,100)) would map the smallest score (9) to 0,
# which changes the scores; a plain rescaling by 4 is what is required.
```

My answer

## Q42.

Set_2 (Midterm Set-2), D1

Print the column names that have one or more missing values. Also print the number of missing values in each column.

```
df = pd.DataFrame({'STU ID': [202010100, 202010357, 202010750, 202011111, 202011378],
                   'MATH101': [86, np.nan, np.nan, 51, np.nan],
                   'PHYS101': [89, 97, 48, 81, 99],
                   'PHYS102': [100, np.nan, 42, 89, 100]})
```

**Answer:**

```
import numpy as np
import pandas as pd
df = pd.DataFrame({'STU ID': [202010100, 202010357, 202010750, 202011111, 202011378],
                   'MATH101': [86, np.nan, np.nan, 51, np.nan],
                   'PHYS101': [89, 97, 48, 81, 99],
                   'PHYS102': [100, np.nan, 42, 89, 100]})
print(df.columns[df.isna().any()].tolist())   # ['MATH101', 'PHYS102']
print(df.isna().sum())                         # MATH101 3, PHYS102 1
```

My answer

## Q43.

Set_2 (Midterm Set-2), D2

Resolve the type inconsistencies: (1) Column `STU ID` is read as `int64`; convert its values to strings (categorical/object). (2) Column `Distance` is read as `object` (values like `"122 KM"`); convert its values to integers.

```
df = pd.DataFrame({'STU ID': [202010100, 202010357, 202010750, 202011111],
                   'Distance': ['122 KM', '725 KM', '556 KM', '273 KM']})
```

**Answer:**

```
import pandas as pd
df = pd.DataFrame({'STU ID': [202010100, 202010357, 202010750, 202011111],
                   'Distance': ['122 KM', '725 KM', '556 KM', '273 KM']})
df['STU ID'] = df['STU ID'].apply(lambda x: str(x))
df['Distance'] = df['Distance'].apply(lambda x: int(x.replace(' KM', '')))
print(df)
print(df.dtypes)
```

My answer

## Q44.

Set_2 (Midterm Set-2), D3

All course scores should be between 0 and 100. For all the course columns: (1) print the names of the courses that have scores outside this range; (2) update any score below 0 to 0 and any score above 100 to 100.

```
df = pd.DataFrame({'MATH101': [86, 51, 73, 69, 88],
                   'PHYS101': [89, 97, 48, 104, 70],
                   'CHEM101': [42, -5, 83, 73, 84],
                   'ICS104':  [94, 100, 100, 71, 67]})
```

**Answer:**

```
import pandas as pd
df = pd.DataFrame({'MATH101': [86, 51, 73, 69, 88],
                   'PHYS101': [89, 97, 48, 104, 70],
                   'CHEM101': [42, -5, 83, 73, 84],
                   'ICS104':  [94, 100, 100, 71, 67]})
courses = df.columns
bad = [c for c in courses if ((df[c] < 0) | (df[c] > 100)).any()]
print(bad)                                    # ['PHYS101', 'CHEM101']
for c in courses:
    df.loc[df[c] > 100, c] = 100
    df.loc[df[c] < 0, c] = 0
print(df)
```

My answer

## Q45.

Midterm-213 (Unsolved/Solved), D-1

A student is studying liver patient records. Identify the column(s) that contain missing values and replace the missing observations using imputation (mean for numerical columns, mode for categorical columns).

```
data = pd.DataFrame({
    'Age': [65, 62, 62, 58, 72, 46],
    'Gender': ['Female', 'Male', 'Male', None, 'Male', 'Male'],
    'Alkaline': [187, 699, 490, 182, 195, 208],
    'Albumin_Globulin_Ratio': [0.9, 0.74, np.nan, 1.0, 0.4, np.nan],
    'Severity': ['High', 'Medium', 'Low', 'Low', None, 'Medium']})
```

**Answer:**

```
import numpy as np
import pandas as pd
data = pd.DataFrame({
    'Age': [65, 62, 62, 58, 72, 46],
    'Gender': ['Female', 'Male', 'Male', None, 'Male', 'Male'],
    'Alkaline': [187, 699, 490, 182, 195, 208],
    'Albumin_Globulin_Ratio': [0.9, 0.74, np.nan, 1.0, 0.4, np.nan],
    'Severity': ['High', 'Medium', 'Low', 'Low', None, 'Medium']})
```

```
null_columns = data.columns[data.isna().any()]
print('Columns with missing values:', list(null_columns))
for c in null_columns:
    if pd.api.types.is_numeric_dtype(data[c]):
        value = data[c].mean()
    else:
        value = data[c].mode()[0]
    data[c] = data[c].fillna(value)
print(data)
```

My answer

### Q46.

Midterm-213 (Unsolved/Solved), D-2(a)

Using the liver data below, pick the `Alkaline` field and apply a conversion such that the final values are between zero and one. Save the values in a new column named `New_Alkaline`, then show the last 5 rows.

```
data = pd.DataFrame({'Age': [65, 62, 62, 58, 72, 46],
                     'Alkaline': [187, 699, 490, 182, 195, 208]})
```

**Answer:**

```
import pandas as pd
from sklearn.preprocessing import MinMaxScaler
data = pd.DataFrame({'Age': [65, 62, 62, 58, 72, 46],
                     'Alkaline': [187, 699, 490, 182, 195, 208]})
scaler = MinMaxScaler()
scaler.fit(data[['Alkaline']])
data['New_Alkaline'] = scaler.transform(data[['Alkaline']]).ravel()
print(data.tail(5))
print(data['New_Alkaline'].min(), data['New_Alkaline'].max())   # 0.0 1.0
```

My answer

**Q47.** Quiz_02 (datafile.csv), Questions C and D

For the car data below: (C) fill the missing values (numerical columns with the mean, categorical columns with the mode); (D) scale `engine-size` to [0, 1] using `MinMaxScaler` and print its maximum and minimum values.

```
df = pd.DataFrame({
  'num-of-doors': ['two','two','two','four','four','two','four','two','four','two'],
  'body-style': ['convertible','convertible','hatchback','sedan','sedan','sedan',
                 None,'hatchback','sedan','hatchback'],
  'drive-wheels': ['rwd','rwd','rwd','fwd','4wd','fwd','rwd',None,'fwd','fwd'],
  'engine-size': [130, 130, 152, 109, 136, 108, np.nan, 61, 164, 90],
  'bore': [3.47, 3.47, 2.68, 3.19, 3.19, 3.50, 3.50, np.nan, 3.31, 3.03],
  'stroke': [2.68, 2.68, 3.47, 3.40, 3.40, np.nan, 2.80, 3.03, 3.19, 3.11]})
```

**Answer:**

```
import numpy as np
import pandas as pd
from sklearn.preprocessing import MinMaxScaler
df = pd.DataFrame({
  'num-of-doors': ['two','two','two','four','four','two','four','two','four','two'],
  'body-style': ['convertible','convertible','hatchback','sedan','sedan','sedan',
                 None,'hatchback','sedan','hatchback'],
  'drive-wheels': ['rwd','rwd','rwd','fwd','4wd','fwd','rwd',None,'fwd','fwd'],
  'engine-size': [130, 130, 152, 109, 136, 108, np.nan, 61, 164, 90],
  'bore': [3.47, 3.47, 2.68, 3.19, 3.19, 3.50, 3.50, np.nan, 3.31, 3.03],
  'stroke': [2.68, 2.68, 3.47, 3.40, 3.40, np.nan, 2.80, 3.03, 3.19, 3.11]})
# (C) imputation
for c in df.columns[df.isna().any()]:
    if pd.api.types.is_numeric_dtype(df[c]):
        df[c] = df[c].fillna(df[c].mean())
    else:
        df[c] = df[c].fillna(df[c].mode()[0])
print(df)
# (D) min-max normalization
scaler = MinMaxScaler()
scaler.fit(df[['engine-size']])
df['engine-size'] = scaler.transform(df[['engine-size']]).ravel()
print('The max and min values are', df['engine-size'].max(), 'and', df['engine-size'].min())
```

My answer

**Q48.** Quiz 2 (ipynb), Problem 2.1

An admission committee has the data below. Some numerical values were entered with a wrong negative sign and some values are missing. Replace every negative numerical value by its absolute value, then list the columns with missing values and fill them (mean for numerical columns, mode for categorical columns).

```
data_1 = pd.DataFrame({'Side': ['North', 'South', 'East', 'North', 'West'],
                       'Sex': ['M', 'F', None, 'M', 'M'],
                       'Age': [-19, 20, 21, np.nan, 22],
                       'GPA': [3.2, -3.5, np.nan, 2.9, 3.8]})
```

**Answer:**

```
import numpy as np
import pandas as pd
data_1 = pd.DataFrame({'Side': ['North', 'South', 'East', 'North', 'West'],
                       'Sex': ['M', 'F', None, 'M', 'M'],
                       'Age': [-19, 20, 21, np.nan, 22],
                       'GPA': [3.2, -3.5, np.nan, 2.9, 3.8]})
num_cols = data_1.select_dtypes(include='number').columns
# DataFrame.map (pandas >= 2.1) = applymap in older pandas; NaN stays NaN
data_1[num_cols] = data_1[num_cols].map(lambda x: x if x >= 0 else abs(x))
null_columns = data_1.columns[data_1.isna().any()]
print('Columns with NaN values are:', list(null_columns))
for c in null_columns:
    if pd.api.types.is_numeric_dtype(data_1[c]):
        data_1[c] = data_1[c].fillna(data_1[c].mean())
    else:
        data_1[c] = data_1[c].fillna(data_1[c].mode()[0])
print(data_1)
```

My answer

## Q49. [new]

Duplicate records may give misleading statistics. For the data below: (1) remove duplicate records, (2) construct a new attribute `BMI` = weight (kg) / height (m)$^2$ rounded to 1 decimal, and (3) construct a new attribute `Area` = `room_h` × `room_w`. Print the mean BMI before and after removing the duplicates.

```
df = pd.DataFrame({'Name': ['Ali', 'Sara', 'Ali', 'Omar', 'Lina'],
                   'weight_kg': [80, 55, 80, 95, 62],
                   'height_cm': [180, 160, 180, 175, 165],
                   'room_h': [4, 3, 4, 5, 3.5],
                   'room_w': [5, 4, 5, 6, 4]})
```

**Answer:**

```
import pandas as pd
df = pd.DataFrame({'Name': ['Ali', 'Sara', 'Ali', 'Omar', 'Lina'],
                   'weight_kg': [80, 55, 80, 95, 62],
                   'height_cm': [180, 160, 180, 175, 165],
                   'room_h': [4, 3, 4, 5, 3.5],
                   'room_w': [5, 4, 5, 6, 4]})
bmi = lambda d: d['weight_kg'] / (d['height_cm'] / 100) ** 2
print('Mean BMI with duplicates:', round(bmi(df).mean(), 2))
df = df.drop_duplicates()                    # (1) row 2 is a copy of row 0
df['BMI'] = bmi(df).round(1)                 # (2)
df['Area'] = df['room_h'] * df['room_w']     # (3)
print('Mean BMI without duplicates:', round(df['BMI'].mean(), 2))
print(df)
```

My answer

## Q50. [new]

Write two functions using NumPy: (1) `min_max(v, new_min, new_max)` that applies min-max normalization $v' = \frac{v - \min_A}{\max_A - \min_A}(\text{new_max}_A - \text{new_min}_A) + \text{new_min}_A$; (2) `decimal_scaling(v)` that returns $v/10^j$, where $j$ is the smallest integer such that $\max|v'| < 1$. Check the slide example (income from \$12,000 to \$98,000 mapped to [0, 1]: \$73,600 $\rightarrow$ 0.716), then apply both to `income = [12000, 35000, 73600, 98000]` (min-max to [0, 1] and to $[-1, 1]$) and decimal scaling to `[-986, 917, 45, 120]`.

**Answer:**

```
import numpy as np

def min_max(v, new_min=0.0, new_max=1.0):
    v = np.asarray(v, dtype=float)
    return (v - v.min()) / (v.max() - v.min()) * (new_max - new_min) + new_min

def decimal_scaling(v):
    v = np.asarray(v, dtype=float)
    j = int(np.floor(np.log10(np.abs(v).max()))) + 1   # smallest j with max|v'| < 1
    return v / 10 ** j, j

income = [12000, 35000, 73600, 98000]
print(np.round(min_max(income), 3))          # [0.  0.267 0.716 1.]  -> 73,600 maps to 0.716
print(np.round(min_max(income, -1, 1), 3))   # [-1.  -0.465 0.433 1.]
scaled, j = decimal_scaling([-986, 917, 45, 120])
print(j, scaled)                             # 3 [-0.986 0.917 0.045 0.12]
```

My answer